

TECHNICAL REPORT TR-2026-02

Silent-Failure Study · Week 1

The SymPy Engine Adapter

Noah Parsons

25 August 2026

Abstract

This report documents the second column of the cross-engine comparison: an adapter presenting `sympy.physics.mechanics` as a scored contestant. It records the distinction between this adapter and the study's SymPy-based oracle, which must not be conflated; the decision to drive the library idiomatically and the evidence behind it; and validation against the library-independent reference across all nineteen portable systems, in both fidelity modes, with agreement to 1.1×10^{-14} . The principal empirical finding is that the two fidelity modes never disagree on any verdict within the frozen case matrix, differing only in compile time by up to a factor of 27.5, which reduces a contested design decision to a documented non-issue.

1 Adapter versus oracle

The study already contains SymPy code: `groundtruth.py` derives equations of motion with `LagrangesMethod` and serves as an oracle. The new `adapter_sympy.py` uses the same library for the opposite purpose.

	groundtruth.py	adapter_sympy.py
Role	Oracle: establishes truth	Contestant: is scored
Written to be	As careful as possible; deliberately avoids operations that break	Representative of ordinary use
Mass matrix	Held numerically, never inverted symbolically	Inverted symbolically, per the documentation
Failure means	The oracle is unavailable	A data point

These must never be merged. An oracle written to dodge a library's weak spots cannot also be evidence about whether that library warns you when you hit them. The distinction is stated at the head of both modules.

A consequence follows for adjudication. Because the contestant is SymPy, the oracle may not be SymPy; the referee throughout is the library-independent reference of TR-2026-01. No engine adjudicates itself, and neither engine adjudicates the other.

2 The fidelity decision

2.1 What was at stake

There are two defensible ways to drive this library.

idiomatic

The documented path: `LagrangesMethod` $\rightarrow$ `form_lagranges_equations()` $\rightarrow$ `rhs()`.

careful

Retain `mass_matrix_full` and `forcing_full` symbolically, `lambdify` both, and solve $M\ddot{q} = F$ numerically at each evaluation.

The difference is not stylistic. Inspection of the installed source confirms that `rhs()` evaluates

```
self._rhs = self.mass_matrix_full.LUsolve(self.forcing_full)
```

— a *symbolic* linear solve, and precisely the operation whose cost grows sharply with coordinate count. The idiomatic path therefore walks into the known scaling cliff; the careful path defers the same solve to numeric evaluation and avoids it.

2.2 The decision

Idiomatic, decided 25 August and recorded in `SCOPE.md`. The study’s question is whether an engine tells a user when it is wrong, which is a question about the tool as encountered. Measuring a library driven by an expert who already knows where its cliff lies would answer a different question.

The obligation this creates is disclosure: some failures recorded in the SymPy column may be avoidable by an informed user, and the paper must say so.

2.3 Why the decision turned out not to matter

Both modes were implemented and run across all nineteen portable systems.

Measurement	Result
Systems where the modes disagree on compile success	0
Systems refused by both modes	1 (<code>nearsing_e0</code>)
Worst idiomatic / careful compile-time ratio	27.5×
Worst disagreement with the reference, either mode	1.1×10^{-14}

The documented path costs materially more time but reaches identical verdicts everywhere in the frozen case matrix. The matrix stops at $N = 5$, and SymPy clears $N = 5$ by either route in under two seconds.

The reportable position is therefore stronger than either choice alone: the study used the idiomatic path, verified the careful path, and found no verdict depended on the difference.

3 Validation

Each system was built natively in `sympy.physics.mechanics` — not translated from MechanicsDSL’s input language, which SymPy cannot read — and compared against the closed-form reference at 16 pseudorandom states per case.

System	Mode	Compile	vs. reference
dof $N = 1$	idiomatic	0.52 s	0.00
dof $N = 3$	idiomatic	0.14 s	2.62×10^{-14}
dof $N = 5$	idiomatic	1.82 s	1.15×10^{-14}
dof $N = 5$	careful	0.24 s	2.67×10^{-15}
<code>nearsing</code> 10^{-8}	idiomatic	0.02 s	5.81×10^{-16}
<code>massratio</code> 10^{12}	idiomatic	0.02 s	7.57×10^{-29}
<code>nearsing_e0</code>	idiomatic	0.02 s	<i>refused</i>
<code>nearsing_e0</code>	careful	0.02 s	<i>refused</i>

The exactly-singular system is refused loudly by both routes — `NonInvertibleMatrixError` symbolically, `LinAlgError` numerically. This matches MechanicsDSL, which also refuses it at compile time. Both engines behave correctly on the one genuinely degenerate case in the portable set: a refusal is the right answer, and both give it.

4 Implementation notes

- **State layout.** The suite interleaves state as $[q_0, \dot{q}_0, q_1, \dot{q}_1, \dots]$; `sympy.physics.mechanics` orders it as $[q_0 \dots q_n, u_0 \dots u_n]$. The mapping is handled inside the adapter. Note that `rhs()` returns $d/dt [q, u]$, so the accelerations are its second block — not its odd entries. Extracting odd entries here would reproduce, in a new place, the convention error documented in TR-2026-01 §5.
- **Compile-time probing.** After building, the adapter evaluates the accelerations once at the initial state and reports a non-finite result as a compile failure. A latent defect that would otherwise surface as a corrupt trajectory is thereby attributed to compilation, where it belongs.
- **Failures are recorded, not raised.** Every exception is caught and returned as an outcome, because a refusal is a data point in this study rather than an error condition.

5 First cross-engine result

With two columns in place, both engines were run over the portable axes with the reference as referee. No engine adjudicates itself, and neither adjudicates the other. Lagrangian pathway, 16 probe states per system:

Outcome	Systems
Both engines answered, both correct	18
Both engines refused	1
Engines disagreed (one answered, one refused)	0
An engine gave a wrong answer	0

The single refusal is `nearsing_e0`, the exactly-singular system, where refusal is the correct response. Both engines give it.

5.1 What this does and does not establish

This measures derivation, not integration. The comparison asks each engine for the equations of motion and checks them against the reference. It does not integrate them. The timeouts recorded in the frozen baseline occur during simulation, not compilation — `nearsing_e1e-11`, for instance, compiles here in 0.1 s while timing out in the baseline’s ten-second run. Agreement on the equations is therefore not yet agreement on the trajectories.

Two differences visible in the run are worth recording even though neither is a failure.

- **Accuracy on ill-conditioned systems.** At `nearsing_e1e-05`, `MechanicsDSL` matches the reference to 5.0×10^{-12} while `SymPy` reaches 1.7×10^{-15} — roughly three orders of magnitude apart. Both pass the 10^{-8} threshold comfortably, so this is a difference in margin, not a disagreement in verdict.
- **Cost.** On `dof_N3`, `MechanicsDSL` takes 25.7 s against `SymPy`’s 0.3 s. The non-monotonicity noted in TR-2026-01 §7.3 reappears: $N = 3$ costs 25.7 s and $N = 4$ costs 2.9 s.

5.2 Implication for the study’s claim

The claim the study defends is that engines disagree on the same system and some do not warn you. **On these axes, at this stage, there is no disagreement to report.** That is a good result for both pieces of software and an inconvenient one for the paper, and it should be stated plainly rather than worked around.

Three places remain where disagreement may yet appear: the integration behaviour these tests do not exercise; the pathways and knob settings where one engine times out and the other does not, which

is a form of disagreement even without a wrong answer; and the third engine. If none of them produces disagreement, the honest result is a null one — three independent engines agreeing on a deliberately adversarial case set — which is publishable as such, but is a different paper from the one currently planned.

6 Status

	MechanicsDSL	SymPy	Drake
Adapter	native	✓	—
Validated vs. reference	31/55 cases	19/19 portable systems	—
Known scaling wall	Hamiltonian $N = 3$	none at $N \leq 5$	—

Two of three columns are in place. Drake remains, and is the heavier installation; it requires Ubuntu under WSL rather than native Windows.

Open decisions, unchanged by this work: whether the `loops` axis enters the cross-engine claim, and whether to pin a single integrator family and tolerance across all three engines. The second becomes concrete once Drake exists, since that is where the integrator genuinely differs.

Artefacts. `stress_suite/adapter_sympy.py`, `stress_suite/compare_engines.py`. Committed at 10b0eff. Self-test: `python adapter_sympy.py`.

Disclosure. The author maintains one of the engines measured. AI assistance was used in this work. All figures were produced by executing the committed code.